

Advanced Database Systems Lecture Guide

Chapter 16: Disk Storage, Basic File Structures, Hashing, and Modern Storage Architectures

Prepared for: Advanced Database Course Lecture

Topic: Disk Storage, File Structure, and Memory Architecture

Date: Wednesday, May 7, 2026

TABLE OF CONTENTS

1. [Introduction](#)
 2. [Storage Hierarchies](#)
 3. [Secondary Storage Devices](#)
 4. [File Organization Concepts](#)
 5. [File Organization Methods](#)
 6. [Hashing Techniques](#)
 7. [RAID Technology](#)
 8. [Modern Storage Architectures](#)
 9. [Frequently Asked Questions & Answers](#)
-

INTRODUCTION

Why is Storage Important?

Databases need persistent storage that can:

- **Store data** reliably over extended periods
- **Retrieve data** quickly and efficiently
- **Handle large volumes** of information that exceed main memory

Key Insight: Understanding storage is crucial because it directly impacts database performance, cost, and reliability.

Three-Level Storage Hierarchy

PRIMARY STORAGE (Main Memory)

- CPU cache, RAM, DRAM
- Fast but expensive & volatile
- Lost when power is off

↓

SECONDARY STORAGE (Disk Storage)

- Magnetic disks, SSDs, flash

- Persistent & relatively affordable
- WHERE DATABASE TABLES ARE STORED



- TERTIARY STORAGE (Archival)
- Removable media, magnetic tape
 - Very slow, cheap, long-term
 - Used for backups & archives

STORAGE HIERARCHIES

Memory Hierarchy Characteristics

Storage Type	Capacity	Speed	Cost	Volatility	Use Case
CPU Cache	KB	Nanoseconds	Very High	Volatile	Immediate operations
RAM/DRAM	MB-GB	Microseconds	High	Volatile	Current program execution
SSD/Flash	GB-TB	Milliseconds	Medium	Non-volatile	Database tables
HDD/Magnetic Disk	GB-TB	Milliseconds	Low	Non-volatile	Database tables
Magnetic Tape	TB-PB	Seconds	Very Low	Non-volatile	Long-term backups

Storage Trade-offs

SPEED vs COST: Fast storage is expensive; cheap storage is slow.

Real Example: In a typical database system:

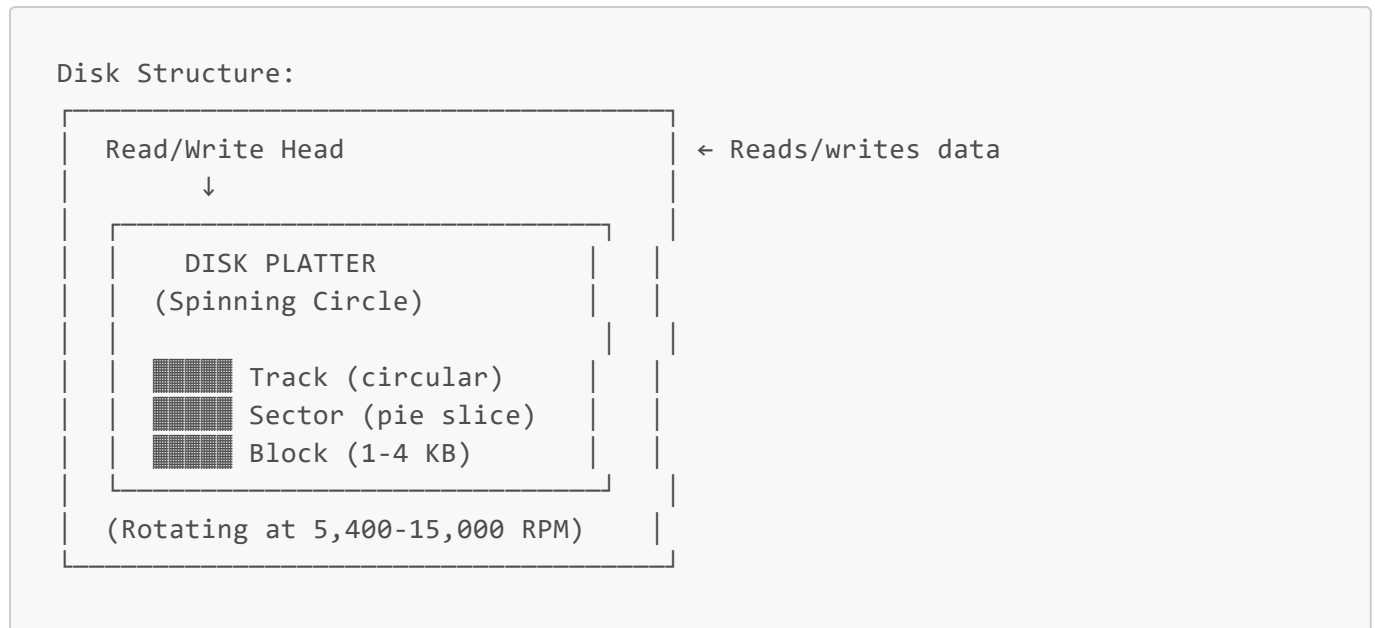
- Data in **cache**: Accessed in nanoseconds (~ 1 ns) – stores ~ 10 MB
- Data in **RAM**: Accessed in microseconds (~ 1 μ s) – stores ~ 16 GB
- Data on **SSD**: Accessed in milliseconds (~ 1 ms) – stores ~ 1 TB
- Data on **HDD**: Accessed in milliseconds (~ 5 - 10 ms) – stores ~ 2 TB
- Data on **Tape**: Accessed in seconds – stores ~ 12 TB

One disk access takes as long as 1 million CPU operations!

SECONDARY STORAGE DEVICES

1. Magnetic Hard Disk Drives (HDD)

How It Works Physically



Key Components

1. Platter

- Circular, rotating disk coated with magnetic material
- Stores actual data using magnetization

2. Tracks

- Concentric circles on the platter
- Like the rings on a tree trunk
- Data organized by track number

3. Sectors

- Pie-shaped wedges within tracks
- Smallest unit that can be read/written
- Typically 512 bytes or 4 KB

4. Blocks (Disk Blocks)

- **PRIMARY UNIT FOR DATABASE I/O**
- Combination of adjacent sectors
- Typical size: 4 KB, 8 KB, or 16 KB
- Created during disk formatting

5. Cylinder

- All tracks at same distance from center across multiple platters
- Important for efficient disk access

Real Example: Reading a Record

Scenario: Access employee record for "John Smith"

Step 1: CPU requests data from block address 2048
Step 2: Disk controller gets request
Step 3: Move read/write head to cylinder containing block
Step 4: Wait for platter to rotate until sector arrives under head
Step 5: Read all 4 KB of data from block
Step 6: Transfer block to buffer in RAM
Step 7: CPU can now access the data

TOTAL TIME: ~5-10 milliseconds (VERY SLOW compared to RAM!)

Disk Access Time Components

Total Time = **Seek Time** + **Rotational Delay** + **Transfer Time** + **Controller Overhead**

- **Seek Time:** Moving head to correct cylinder (2-5 ms)
- **Rotational Delay:** Waiting for sector to arrive (0-3 ms average)
- **Transfer Time:** Reading the data (0.1-0.5 ms)

2. Solid State Drives (SSD)

How SSDs Differ

HDD vs SSD:

HDD (Mechanical):

- Moving parts (spinning platter, moving head)
- Slower (~5-10 ms access)
- Cheaper per GB
- Subject to physical failure from shock

SSD (Electronic):

- No moving parts (all flash memory)
- Faster (~0.1-0.5 ms access)
- More expensive per GB
- More reliable (no mechanical failure)
- Better power efficiency

SSD Architecture

SSD Controller

↓

Interconnected Flash Memory Cards



Data stored electronically (not magnetically)

Advantages:

- No data fragmentation (data doesn't scatter over disk)
- Consistent access times (no seek time variability)
- Much faster than mechanical disks

Disadvantages:

- Higher cost than HDDs
- Limited number of write cycles (eventually wears out)
- Write speeds sometimes slower than reads

3. Magnetic Tape Storage

Use Cases

- **Backups:** Create copies of entire databases
- **Archives:** Long-term storage of infrequently accessed data
- **Disaster Recovery:** Store backup tapes offsite

Key Characteristics**Access Method:** Sequential only

- Must scan from beginning to find a record
- Like a VCR tape vs random access of a CD

Example:

```
Finding record at position 1000 on tape:  
- Must read records 1, 2, 3, ... 999 first  
- Takes seconds or minutes depending on tape length  
- Compare to disk: direct access in milliseconds
```

Storage Capacity: Massive

- Single tape: ~12 TB
- Tape library: Petabytes

Cost: Very cheap per GB

- Good for archival data

FILE ORGANIZATION CONCEPTS

Key Terms

1. File

- Collection of records of same type
- Example: EMPLOYEE file with all employee records

2. Record

- Collection of related data fields
- Example: One employee record (ID, Name, Salary, Department)

3. Block

- Unit transferred between disk and memory
- Contains one or more complete/partial records

4. Field

- Individual data item
- Example: Employee ID, Salary

Record Structure

EMPLOYEE Record:

Emp_ID	Name	Salary	Department
E001	John Smith	50,000	Sales
E002	Mary Johnson	60,000	Engineering
E003	Bob Wilson	55,000	HR

Record Blocking

Concept: How Many Records Fit in a Block?

Example Calculation:

Block size: 4 KB = 4096 bytes

Record size: 200 bytes

Blocking factor (bfr) = $4096 / 200 = 20$ records per block

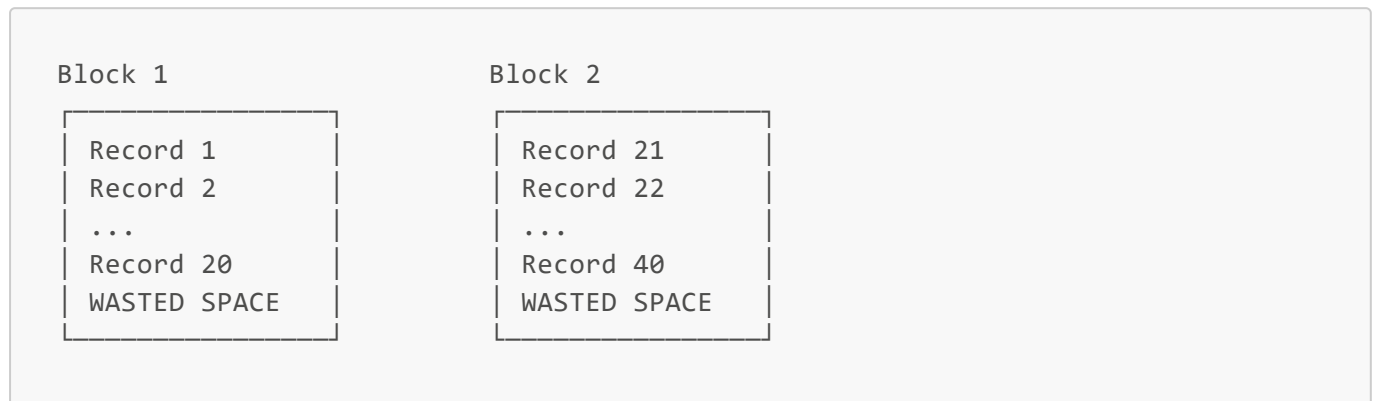
This means:

- 20 employee records fit in one disk block
- One disk read gets you 20 records
- This is MUCH more efficient than reading 1 record per block

Spanned vs Unspanned Records

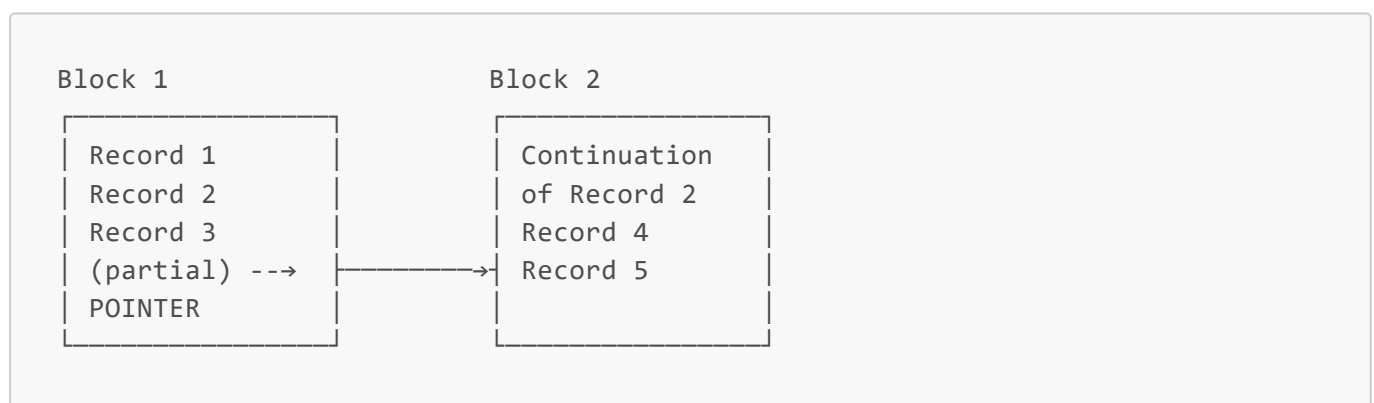
Unspanned Records:

- Records cannot cross block boundaries
- Simpler implementation
- Some wasted space at end of block



Spanned Records:

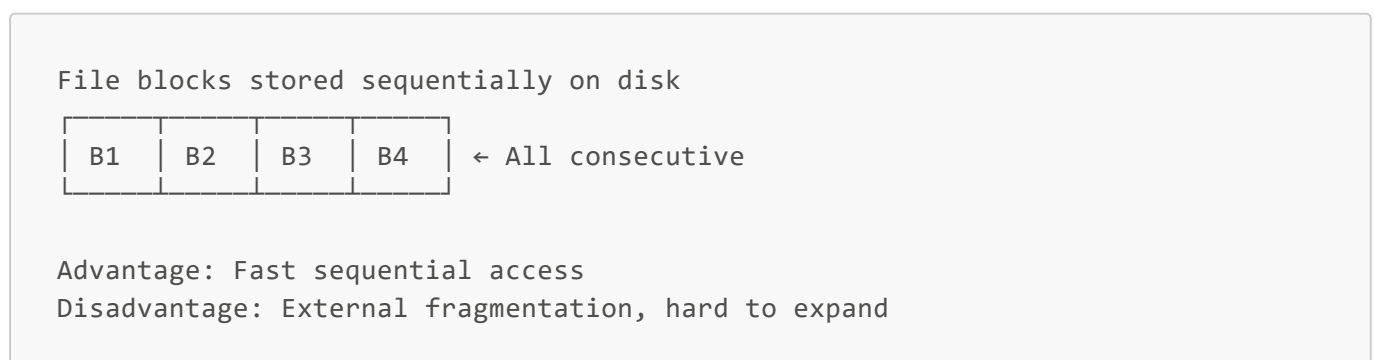
- Large records can cross block boundaries
- Pointer at end of block shows continuation location
- More complex but space-efficient



File Allocation Strategies

When a file grows, how do we allocate new disk blocks?

1. Contiguous Allocation



2. Linked Allocation

File blocks can be scattered, with pointers connecting them



Advantage: Easy to grow, no fragmentation

Disadvantage: Slow for random access (must follow pointers)

3. Indexed Allocation

Index block contains all addresses



Advantage: Fast random access, easy to grow

Disadvantage: Extra index overhead

FILE ORGANIZATION METHODS

1. Heap Files (Unordered Records)

How It Works

- Records inserted in order received
- No particular organization
- Random order on disk

Example

Insert order: E003, E001, E005, E002, E004

File on disk:

E003 (Bob)	E001 (John)	E005 (Carol)	Block 1
E002 (Mary)	E004 (David)	(empty)	Block 2

Efficiency Analysis

Operation	Performance	Why?
Insert	$O(1)$ - Very Fast	Add to end of file
Search	$O(n)$ - Slow	Must read all blocks
Delete	$O(n)$ - Slow	Must find record first

Real Example:

```

EMPLOYEE file with 100,000 employees
If 20 employees per block:
- Need 5,000 blocks on disk
- Average search: 2,500 blocks must be read
- At 5ms per block = 12.5 seconds for one search!

```

Use Cases

- Initial data loading
- Temporary tables
- When no particular access pattern is dominant

2. Ordered (Sorted) Files

How It Works

- Records sorted by a key field (ordering key)
- Maintains sorted order as records inserted/deleted

Example

Sorted by Employee ID:

E001 (John)	E002 (Mary)	E003 (Bob)	Block 1
E004 (David)	E005 (Carol)	E006 (Tom)	Block 2

Efficiency Analysis

Operation	Performance	Method
Insert	$O(\log n)$ search $O(n)$ reorganize	Binary search to find position, shift records
Search	$O(\log n)$	Binary search (very fast!)

Operation	Performance	Method
Delete	$O(\log n)$ search $O(n)$ reorganize	Find and mark or reorganize

Real Example with Binary Search:

Finding E00547 in 1,000,000 employees:

Heap file: ~2,500 block reads (average)

Sorted file with binary search:

$\log_2(1,000,000) \approx 20$ block reads

SPEEDUP: 125x faster!

Advantages

- Very efficient for range queries
- Good for sequential access
- Binary search is extremely fast

Disadvantages

- Expensive insertion/deletion (must maintain order)
- High overhead for frequent updates

Use Cases

- Read-heavy systems
- Batch processing of data
- When queries often request ranges

3. Hash Files

Core Concept

Apply a **hash function** to key field to directly compute disk block address.

Hash Function Example:

$h(\text{key}) = (\text{key} \bmod 10)$

Employee E001 $\rightarrow h(001) = 001 \bmod 10 = 1 \rightarrow \text{Block 1}$

Employee E023 $\rightarrow h(023) = 023 \bmod 10 = 3 \rightarrow \text{Block 3}$

Employee E047 $\rightarrow h(047) = 047 \bmod 10 = 7 \rightarrow \text{Block 7}$

Employee E032 $\rightarrow h(032) = 032 \bmod 10 = 2 \rightarrow \text{Block 2}$

Efficiency

Operation	Performance	Why?
Search (equality)	$O(1)$ - FASTEST!	Direct computation
Insert	$O(1)$ - FASTEST!	Direct computation
Range queries	$O(n)$ - Slow	Hash doesn't preserve order
Sequential scan	$O(n)$ - Slow	Records scattered

Real Example

Find: Employee with ID E00523

Unordered file: Binary search through ~100,000 blocks

Hash file:

1. Apply hash: $h(E00523) = 23 \bmod 10 = 3$
2. Go directly to block 3
3. Access time: 1 disk read (~5ms)

SPEEDUP: 1,000,000x faster for direct lookup!

Problem: Collisions

Collision: When two keys hash to the same block address

$h(001) = 001 \bmod 10 = 1$

$h(011) = 011 \bmod 10 = 1 \leftarrow \text{COLLISION!}$

Block 1

E001 (John)	E011 (Sarah)	(overflow)
-------------	--------------	------------

Collision Resolution Strategies

1. Open Addressing (Linear Probing)

If block is full, place in next available block

$h(011) = 1$, but block 1 full

→ Try block 2, also full

→ Try block 3, free! → Insert there

Retrieval: Go to block 1, not found, check 2, not found, check 3, FOUND!
Problem: Can create "clustering" - groups of occupied blocks

2. Chaining

Colliding records linked together

Block 1

Block 2



Pointer leads to next record with same hash

3. Multiple Hashing

Try second hash function if first causes collision
 $h_1(011) = 1$ (full)
 $h_2(011) = 5$ (available) → Insert there

Dynamic Hashing

Problem: Static hash tables waste space or overflow

Solution 1: Extendible Hashing

Dynamically expands hash table as file grows

- File performance stays consistent
- No performance degradation
- Extra overhead for directory management

Solution 2: Linear Hashing

Gradually increases bucket count without complete rehashing

- Allows file to expand and shrink
- No directory needed
- Smoother growth pattern

Use Cases

- **Perfect for:** Equality searches (WHERE ID = 123)
- **Poor for:** Range queries (WHERE Salary > 50000)
- **Good for:** Authentication systems, lookup tables
- **Bad for:** Reporting, analytics

HASHING TECHNIQUES

Internal vs External Hashing

Internal Hashing:

- Hash table in main memory
- Used for temporary data structures
- Very fast
- Limited by RAM size

External Hashing:

- Hash file on disk
- Bucket = one or more disk blocks
- Hash function maps key to bucket number
- Bucket number converted to disk block address

Hash Function Design

Requirements for good hash function:

1. UNIFORM DISTRIBUTION
 - Keys should hash to all buckets equally
 - Bad: all keys hash to same bucket
 - Good: keys spread evenly
2. DETERMINISTIC
 - Same key always produces same hash
 - No randomness
3. EFFICIENT
 - Computable in $O(1)$ time
4. SIMPLE
 - Avoid complex calculations

Example Hash Functions

1. Division Method:
 $h(k) = k \bmod m$
 - Simple, fast
 - m should be prime
 - Example: $h(47) = 47 \bmod 10 = 7$
2. Multiplication Method:
 $h(k) = \lfloor m \times (k \times A \bmod 1) \rfloor$
 - A is constant (0.618...)

- More uniform distribution
- More complex

3. Folding Method:

Divide key into parts, sum them

- For key 123456789:
- Fold: $123 + 456 + 789 = 1368$
- $h(k) = 1368 \bmod 10 = 8$

RAID TECHNOLOGY

What is RAID?

RAID = Redundant Array of Independent Disks

Goal: Improve speed and reliability by using multiple disks

Key Concepts

1. Data Striping

- Spread data across multiple disks
- Read/write to multiple disks simultaneously
- Increases transfer rates

Data: ABCDEFGH spread across 3 disks

Disk 1: A D G

Disk 2: B E H

Disk 3: C F

Reading ABCDEFGH: 3 disks working simultaneously!
Speed is 3x faster than single disk

2. Redundancy

- Store duplicate/parity data
- Protect against disk failure
- Can recover lost data

Original: ABCD

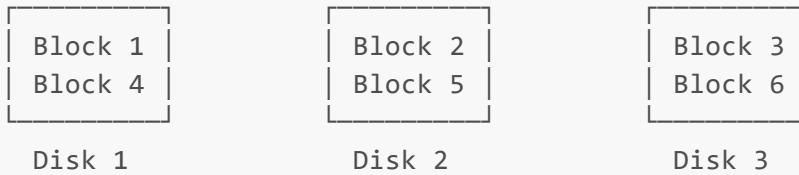
Redundant copy on different disk: ABCD

If Disk 1 fails:

- Restore from Disk 2
- No data loss!

RAID Levels

RAID 0 (Striping, No Redundancy)



Advantages:

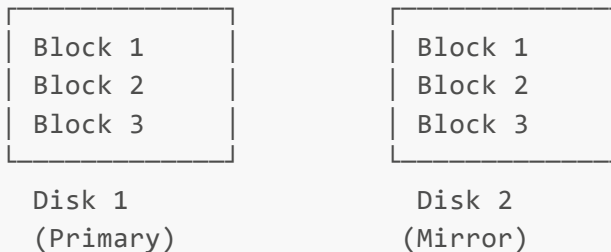
- 3x faster read/write (parallel)
- Maximum capacity

Disadvantages:

- NO redundancy
- If any disk fails → ALL data lost!
- Risk higher than single disk

Use Case: Temporary data, cache, non-critical data

RAID 1 (Mirroring)



Advantages:

- Complete redundancy
- Instant failover
- Good read performance (can read from either)

Disadvantages:

- 50% capacity lost (need duplicate)
- More expensive
- Slower write (must write to both)

Use Case: Critical data, financial systems

RAID 5 (Striping with Distributed Parity)



Block 1	Block 2	Block 3	Parity 1
Block 5	Parity 2	Block 4	Block 6
Parity 3	Block 7	Block 8	Block 9
Block 10	Block 11	Parity 4	Block 12

Key: Parity blocks spread across all disks

How Parity Works:

Parity = Block1 XOR Block2 XOR Block3

If Block 2 fails:

Block2 = Block1 XOR Block3 XOR Parity

Can RECOVER any lost block!

Advantages:

- Good balance: performance + redundancy
- Can survive 1 disk failure
- Only ~25% capacity lost
- Preferred for most applications

Disadvantages:

- More complex
- Write penalty (must calculate parity)
- Recovery slower than RAID 1

Use Case: Most common for production databases

RAID 6 (Dual Parity)

Like RAID 5 but with TWO parity blocks

Advantages:

- Can survive 2 disk failures
- Better for large disk arrays

Disadvantages:

- More complex
- Higher write overhead
- Slower than RAID 5

Use Case: Enterprise environments, mission-critical systems

RAID Comparison Table

Level	Method	Redundancy	Performance	Capacity	Failures	Use Case
0	Striping	None	Fastest	Full	0 = Total loss	Cache, temp
1	Mirroring	Full copy	Fast	50%	1 okay	Critical data

Level	Method	Redundancy	Performance	Capacity	Failures	Use Case
5	Striping + Parity	1/n	Good	~75%	1 okay	Databases
6	Striping + Dual Parity	2/n	Moderate	~67%	2 okay	Enterprise

MODERN STORAGE ARCHITECTURES

1. Storage Area Networks (SAN)

What is it?

Traditional Setup:

Server 1 → Disk 1

Server 2 → Disk 2

Server 3 → Disk 3

(Each server has own storage)

SAN Setup:

Server 1 ↴

Server 2 } → High-Speed Network → Shared Storage Pool

Server 3 ↴

All servers access shared storage devices

Characteristics

- Dedicated high-speed network (Fiber Channel)
- Multiple servers share storage
- Centralized management
- Professional-grade reliability

Protocols

1. iSCSI (Internet SCSI)

- SCSI commands over IP network
- Uses standard Ethernet
- More affordable than Fiber Channel
- Slower but good for most applications

2. Fiber Channel over IP (FCIP)

- Fiber Channel commands wrapped in IP packets
- Connects distant data centers
- Geographic redundancy

3. Fiber Channel over Ethernet (FCoE)

- Fiber Channel over Ethernet network
- Reduces network complexity

Real Example

```
Bank's Data Center:  
- 50 database servers  
- Each needs access to customer data  
- Single shared SAN with RAID 6  
- If one server fails, others continue accessing data  
- All transactions logged for recovery
```

2. Network-Attached Storage (NAS)

How It's Different

```
SAN: Block-level storage (SCSI commands)  
      Server must manage file system  
  
NAS: File-level storage (NFS/SMB protocols)  
      NAS device manages file system
```

Advantages

- Simpler to set up
- File-sharing capabilities
- Better for document storage
- Supports multiple protocols

Real Example

```
Company with multiple offices:  
- Headquarters has central file server (NAS)  
- Office 1 mounts: \\storage\documents  
- Office 2 mounts: \\storage\documents  
- All offices see same files  
- Centralized backup  
  
This is like Dropbox for enterprise
```

3. Automated Storage Tiering

Concept: Hot/Cold Data

HOT DATA (Frequently Used):

- Stored on fastest storage (SSD)
- Fast access for reports, dashboards
- Example: Sales data from last month

WARM DATA (Occasionally Used):

- Stored on medium speed (HDD)
- Example: Sales data from last year

COLD DATA (Rarely Used):

- Stored on cheapest storage (Tape, Archive)
- Example: Historical data from 5 years ago

How It Works

System continuously monitors access patterns:

Day 1: New customer data created

- Placed on SSD (auto-tiering)

Month 1: Customer active, frequent queries

- Stays on SSD

Year 2: Customer inactive, rare queries

- System moves to HDD automatically

Year 5: Archive only

- System moves to tape automatically

User doesn't care where data is!

System optimizes for performance AND cost

Real-World Example: Retail Database

Walmart Product Database:

HOT (SSD):

- Current week's sales
- Active promotions
- Real-time inventory

WARM (HDD):

- Last year's sales
- Past promotions
- Archive inventory

COLD (Tape):

- 5-year historical data
- For regulatory compliance
- Accessed maybe 1x/year

Result: 90% of queries on 10% of data on fastest storage

4. Object-Based Storage

Traditional Approach (File/Block)

File: A sequence of bytes

0x4A 0x6F 0x68 0x6E 0x20 0x53 0x6D	"John Smith"
------------------------------------	--------------

Problem: System knows nothing about content meaning

Object-Based Approach

Object: Data + Metadata + Unique ID

OBJECT
Unique ID: obj-12345
DATA: "John Smith", "Sales", "50000"
METADATA: Created: 2026-05-04 Owner: HR_Department Classification: Confidential Retention: 7 years Access_Log: [...]
GLOBAL ID: urn:employee:12345

Advantages

- **Scalability:** Petabytes of unstructured data
- **Intelligence:** Object carries its own metadata
- **Global ID:** Access from anywhere
- **Flexible:** Any type of data (documents, images, videos)

Use Cases

- Cloud storage (AWS S3 - object storage!)
- Media repositories (images, videos)
- Backup/archive systems
- IoT sensor data

FREQUENTLY ASKED QUESTIONS & ANSWERS

Q1: Why can't we just use RAM for everything?

Answer:

RAM is volatile and expensive:

```
RAM: $10-15 per GB
    Access time: 100 nanoseconds
    Volatile (lost on power failure)

Disk: $0.01-0.1 per GB
    Access time: 5-10 milliseconds
    Persistent (survives power failure)
```

Real numbers:

- 1 TB of RAM = \$12,000
- 1 TB of SSD = \$100
- 1 TB of HDD = \$30

For a company with 100 TB database, RAM would cost \$1.2 million vs \$3,000 for HDDs!

Plus, even the fastest computers have limited RAM. A laptop has 16 GB max, but may need to access 500 GB database.

Q2: If SSDs are faster than HDDs, why not use only SSDs?

Answer:

SSDs have limitations:

Factor	HDD	SSD
Cost per GB	\$0.02-0.05	\$0.08-0.15
Lifespan	10+ years	5-7 years
Write cycles	Unlimited	~3,000-5,000 cycles
Capacity	Up to 20 TB easily	Typically up to 4 TB

Real example:

Database: 100 TB data

With HDDs:

Cost: \$3,000

Can store indefinitely

With SSDs:

Cost: \$15,000 (5x more!)

Each sector dies after 5,000 writes

High-write databases fail quickly

Smart approach:

- SSD: Hot data (last month) - \$1,000 for 10 TB
- HDD: Archive data (older) - \$2,000 for 90 TB
- Automatic tiering moves between them
- Total cost: \$3,000, performance + reliability

Q3: What happens if a disk fails in RAID 5?**Answer:**

RAID 5 is designed for exactly this scenario:

RAID 5 with 4 disks, 1 fails:

Step 1: Failure detected, system switches to degraded mode

Step 2: Parity blocks used to recalculate lost data

Step 3: Lost blocks reconstructed in memory

Step 4: User's query still works (transparent!)

Example:

Disk 2 fails:

Block 5 lost, Parity 2 survives

New Block 5 = Block 1 XOR Block 4 XOR Parity 2

Time to recover: Hours (system reconstructs all blocks)

Data loss: ZERO

Best practice: Replace disk immediately

When replacement arrives, new blocks written, redundancy restored

What if 2 disks fail?

RAID 5: Game over - 2 or more failures = Total loss

RAID 6: Still okay - designed for 2 failures

Can recover from 2 disk failures

Q4: When should I use ordered files vs hash files?

Answer:

Depends on query patterns:

USE ORDERED FILES when:

- ✓ Range queries common
"Find all salaries between 50K-100K"
Binary search on sorted key = fast
- ✓ Sequential access needed
"Process all employees in order"
- ✓ Updates common
Even though inserts/deletes slow,
if queries much more frequent, worth it
- ✓ Multiple sort orders needed
Can create sorted file per key

USE HASH FILES when:

- ✓ Exact equality searches common
"Find employee E00523"
Direct computation = fastest possible
- ✓ High-volume inserts
No reorganization overhead
- ✓ Main queries on one field
Hash on that field
- ✓ No need for ordering
Just need fast lookup

EXAMPLE:

Social Media System:

- User login: Hash on UserID (instant lookup)
- Browse timeline: Use different index (ordered by time)
- Search users by name: Use different index (ordered by name)

(One table, multiple different file structures for different access patterns)

Q5: How does the operating system handle buffer management?

Answer:

When database needs to read/write, OS manages buffers:

PROCESS:

1. Database requests block 2048
2. OS checks: Is it in buffer cache?
- 3a. YES → Return immediately (cache hit)
- 3b. NO → Read from disk, put in buffer

BUFFER REPLACEMENT STRATEGY (when buffer full):**Strategy 1: LRU (Least Recently Used)**

- Remove buffer not used longest time
- Best for random access

Strategy 2: Clock Policy

- Simpler than LRU
- Almost as good performance

Strategy 3: FIFO (First-In-First-Out)

- Remove oldest buffer
- Simple but less effective

EXAMPLE:

Buffer slots 1-4 (each 4KB):

[E001 block] [E002 block] [E005 block] [empty]

Access E003 block (not in cache):

- OS reads E003 from disk
- Needs space in buffer
- LRU: E001 was used longest ago
- Remove E001, place E003
- E001 back to disk if modified

[E003 block] [E002 block] [E005 block] [empty]

****The larger buffer pool, the better (more cache hits)****

Q6: How does file blocking factor affect performance?

****Answer:****

Blocking factor (records per block) is critical:

EXAMPLE: 1 million employee records

SCENARIO 1: Large blocking factor

Block size: 4 KB

Record size: 100 bytes

Blocking factor: 40 records/block

Total blocks needed: $1,000,000 / 40 = 25,000$ blocks

Linear scan: Read 25,000 blocks

SCENARIO 2: Small blocking factor

Record size: 100 bytes → split into 2 fields per block

Blocking factor: 20 records/block

Total blocks needed: $1,000,000 / 20 = 50,000$ blocks

Linear scan: Read 50,000 blocks

DIFFERENCE:

Scenario 1: $25,000 \times 5\text{ms} = 125$ seconds

Scenario 2: $50,000 \times 5\text{ms} = 250$ seconds

= TWICE AS SLOW!

LESSON: Choose large block size to fit more records

But balance with:

- Buffer pool size (can't buffer too many huge blocks)
- Update frequency (more records per block = more I/O for updates)
- Memory constraints

Q7: What's the difference between seek time and rotational delay?

****Answer:****

Two components of disk access time:

SEEK TIME: Moving read/write head

Disk structure:

```

┌───────────┐
| Track 0 | ← Head starts here
| Track 5 |
| Track 10 | ← Need to go here

```

| Track 15 |

Action: Move head from Track 0 to Track 10

Time: ~2-5 milliseconds (depends on distance)

ROTATIONAL DELAY: Waiting for sector to arrive

Disk rotating at 7200 RPM (typical):

Time for 1 full rotation = $60\text{sec} / 7200 = 8.3\text{ms}$

Average wait (sector could be anywhere) = 4.15ms

EXAMPLE:

Reading block at Track 10, Sector 45:

1. Seek phase: Move head to Track 10
Time: 3ms
2. Rotation phase: Platter rotates until Sector 45 under head
Time: 4ms (average, up to 8.3ms worst case)
3. Transfer: Read 4KB at 10MB/sec
Time: 0.4ms

TOTAL: $3 + 4 + 0.4 = 7.4\text{ms}$

Key insight: Rotational delay often LARGER than seek time!

This is why:

- Sequential access is good (no seek)
- Proper data layout saves rotational delay
- SSDs have no rotational delay!

Q8: How do you calculate required storage capacity?

****Answer:****

Estimate based on data size and growth:

FORMULA:

Total Capacity = (Raw Data Size $\times$ 1.3) + Backups + Logs

Components:

1. RAW DATA SIZE
Employees table:

- 1 million employees
- 200 bytes per record
- Size = $1\text{M} \times 200 = 200\text{ MB}$

Customers table:

- 50 million customers
- 500 bytes per record
- Size = $50\text{M} \times 500 = 25\text{ GB}$

Orders table:

- 500 million orders
- 100 bytes per record
- Size = $500\text{M} \times 100 = 50\text{ GB}$

Total raw data: 75 GB

2. OVERHEAD ($\times 1.3$)

Indexes: 20-30% of data size

System structures: 5-10%

Fragmentation: 5%

Overhead: $75\text{ GB} \times 0.3 = 22.5\text{ GB}$

3. BACKUP COPIES

Daily backups $\times 7$ days: $75 \times 7 = 525\text{ GB}$

Weekly archival: $75 \times 4 = 300\text{ GB}$

Backups: 825 GB

4. TRANSACTION LOGS

Average 10% of daily data change

10 days log retention: $75 \times 0.1 \times 10 = 75\text{ GB}$

TOTAL: $75 + 22.5 + 825 + 75 = 997.5\text{ GB} \approx 1\text{ TB}$

PLUS: Growth projection

If growing 20% per year:

Year 1: 1 TB

Year 2: 1.2 TB

Year 3: 1.44 TB

Purchase for Year 3: 2 TB (good margin)

REAL EXAMPLE: Netflix

Q9: Why is database I/O often the bottleneck?

****Answer:****

Speed differences are dramatic:

CPU operation: 1 nanosecond

RAM access: 100 nanoseconds (100× slower)

SSD access: 100,000 nanoseconds (100,000× slower)

HDD access: 10,000,000 nanoseconds (10,000,000× slower!)

EQUIVALENT TIMES (if CPU operation = 1 second):

1 second: CPU operation

100 seconds: RAM access (~1.5 minutes)

100,000 seconds: SSD (~1 day)

10,000,000 seconds: HDD (~115 days)

QUERY EXAMPLE:

Finding one record in unsorted file:

Binary search:

- CPU: 20 operations (20 nanoseconds)
- I/O: 20 disk reads (200 milliseconds)

WAIT TIME: 200ms (10 million times slower!)

This is why:

1. Database designers minimize I/O operations
2. Caching is so important
3. Query optimization focuses on disk reads
4. Indexing is critical

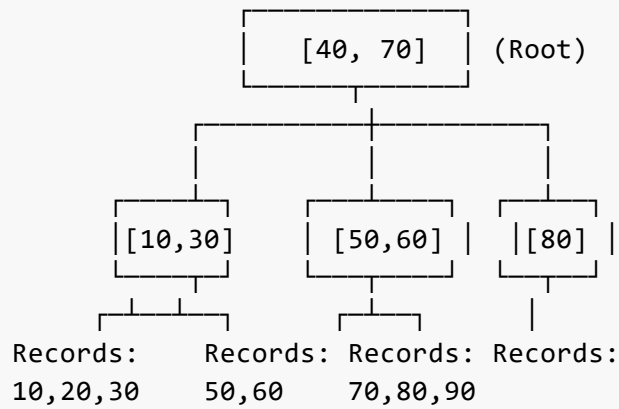
ONE DISK READ takes as long as 1,000,000 CPU operations!

Q10: How does a B-tree index improve search performance?

****Answer:****

B-tree balances performance for range queries and equality searches:

TREE STRUCTURE:



SEARCH: Find all salaries 55-75

1. Start at root [40, 70]
2. 55 falls between 40 and 70, go middle
3. At [50, 60], 55 falls after, go right
4. At [70, 80, 90], 70 is in range
5. Collect records: 55, 60, 70, 75

EFFICIENCY:

- One root read
- One level read
- One leaf read
- Total: 3 disk reads for entire range!

LINEAR SCAN:

- Read 1,000,000 records from disk
- Check each one
- Total: 50,000 disk reads

B-TREE SPEEDUP: 16,667× faster!

This is why databases always index for range queries

Q11: What's the difference between backup and archive?

Answer:

Often confused, but different purposes:

BACKUP:

Purpose: Recover from failure

Frequency: Daily, weekly

Retention: 1 week to 1 month

Access: Fast recovery time critical

Technology: RAID mirrors, fast disks

Automation: Automatic, regular schedule

Scenario: Database corrupted by software bug

- Restore from backup from 1 day ago
- Lose 1 day of changes
- System back up quickly

ARCHIVE:

Purpose: Long-term storage, compliance

Frequency: Once created, never updated

Retention: Years (often 7+ years legally)

Access: Slow access acceptable

Technology: Tape, object storage

Automation: Move old data automatically

Scenario: Regulatory compliance requires 7-year history

- Store 2019 records on tape
- Access maybe once per year for audits
- Cheap storage

COMPARISON:

	Backup	Archive
Cost	Higher	Very low
Speed	Fast	Slow OK
Frequency	Regular	One-time
Purpose	Recovery	Compliance

REAL EXAMPLE:

Bank:

- Every night: Backup current data to SSD RAID
(Can restore in hours if problem)
- Every month: Archive old records to tape
(Required to keep 7 years for regulations)
- Old archives: Keep in secure offsite location
(Physical security for disaster recovery)

Q12: How does data consistency relate to disk I/O?

****Answer:****

Disk I/O timing affects consistency:

SCENARIO: Transfer money from checking to savings

Step 1: Read checking balance: \$1,000

Step 2: Read savings balance: \$500

Step 3: Subtract \$200 from checking: \$800

Step 4: Add \$200 to savings: \$700

WITHOUT disk considerations:

- All updates in memory immediately
- All reads happen in order
- Consistent

WITH disk I/O delays:

Timeline:

T=0 Step 1: Read checking (\$1,000)

T=5ms Step 3: Reduce to \$800, write to disk (in progress...)

T=6ms CRASH! Power failure

Savings write never happened

Result: \$200 disappeared! Inconsistent state.

SOLUTION: Transaction Logging

Write log BEFORE making changes:

Log entry: "Transfer 200 from checking to savings"

Wait for log disk write complete

Then: Update checking balance

Then: Update savings balance

Then: Write commit to log

If crash:

- Check log during recovery
- "Transfer incomplete, rollback"
- Restore both accounts to original state

Or:

- "Transfer completed"
- Apply both changes after recovery

LESSON: Disk I/O ordering critical for data integrity

This is why databases have:

1. Write-ahead logging (WAL)
2. Transaction coordinators
3. Commit protocols
4. Careful disk scheduling

Q13: What's the impact of record size on database performance?

****Answer:****

Record size affects everything:

IMPACT ON BLOCKING FACTOR:

Block: 4096 bytes

Record 1: 100 bytes → 40 records/block

Record 2: 200 bytes → 20 records/block

Record 3: 500 bytes → 8 records/block

Record 4: 4000 bytes → 1 record/block

Impact on scan performance:

Scan 1 million records:

Record size 100: $1\text{M}/40 = 25,000$ reads $\times 5\text{ms} = 125$ seconds

Record size 200: $1\text{M}/20 = 50,000$ reads $\times 5\text{ms} = 250$ seconds

Record size 500: $1\text{M}/8 = 125,000$ reads $\times 5\text{ms} = 625$ seconds

DIFFERENCE: $5\times$ slower for $5\times$ larger records!

BEST PRACTICES:

1. Include only necessary fields
2. Use appropriate data types
(INT instead of VARCHAR(50) for IDs)
3. For frequent queries, create smaller record sets
4. Normalize to reduce record size

Example: Customer record

BAD:


```
CREATE TABLE Customers (  
  ID INT,  
  Name VARCHAR(100),  
  Address VARCHAR(500),  
  Phone VARCHAR(20),  
  Email VARCHAR(100),  
  Birthday DATE,  
  Biography TEXT, ← 1MB per customer!  
  ProfilePhoto BLOB, ← 5MB per customer!  
  NotesAboutCustomer TEXT ← 10KB per customer  
);
```

Record size: ~6-7MB each

1 record per block

Queries very slow

GOOD:

```
Table: Customers (primary)  
  ID, Name, Address, Phone, Email, Birthday  
  Size: ~150 bytes  
  26 records per block  
  
Table: CustomerBio (for detailed info)  
  CustomerID, Biography, ProfilePhoto  
  Referenced only when needed  
  
Result: Main queries 40x faster
```

Q14: How do you choose between different RAID levels?

Answer:

RAID decision tree:

```
QUESTION 1: Is this critical data?  
  
NO → Use RAID 0 (striping only)  
  - Maximum performance  
  - Maximum capacity  
  - Cost: Lowest  
  - Risk: Any disk failure = total loss  
  - Use for: Temp tables, cache, logs  
  
YES → Go to Question 2
```

QUESTION 2: How many reads vs writes?

MOSTLY READS (Reports, Analytics) → RAID 5

- Good performance
- 1 disk failure protection
- Cost: Moderate
- Use for: Data warehouses, archives

MOSTLY WRITES (Transactions) → RAID 1

- Good write performance
- Instant failover
- Cost: High (2× disks)
- Use for: Transaction logs

MIXED (General databases) → RAID 5

- Balanced approach
- Most common choice
- Cost: Moderate

QUESTION 3: How important is recovery time?

CRITICAL (Bank, Hospital) → RAID 6 + RAID 1 Hybrid

- Survive 2 failures
- Hot spares on standby
- Automatic reconstruction
- Cost: Very high
- RTO (Recovery Time Objective): minutes

HIGH (E-commerce) → RAID 5 + Hot spares

- Survive 1 failure
- Spare disk ready to rebuild
- Cost: Moderate
- RTO: Hours

MEDIUM (Startup) → RAID 5

- Survive 1 failure
- Accept longer recovery
- Cost: Moderate
- RTO: Days acceptable

REAL EXAMPLES:

Netflix (streaming):

- RAID 0 for streaming cache (performance critical)
- RAID 5 for user data (important but can handle hour downtime)

Bank (transactions):

- RAID 1 for transaction database (zero loss acceptable)
- RAID 6 for archives (double protection)
- Hot spares + automatic failover

University (data warehouse):

- RAID 5 (cost effective, protect research)
- Large backups (data is researcher's life work)

Q15: What's the relationship between cache hit rate and database performance?

Answer:

Cache is EVERYTHING for database performance:

```
CPU operates at nanoseconds
Every cache MISS = wait milliseconds (1,000,000x slower)

CACHE HIT RATE = % of requests found in cache

Example: 80% hit rate

Out of 1,000 queries:
- 800 hit cache: 800 × 0.1ms = 80ms
- 200 miss cache: 200 × 5ms = 1,000ms

Total: 1,080ms for 1,000 queries

If hit rate only 50%:
- 500 hit cache: 500 × 0.1ms = 50ms
- 500 miss: 500 × 5ms = 2,500ms

Total: 2,550ms

IMPACT: 2.4x SLOWER with lower cache hit rate!

HOW TO IMPROVE CACHE HIT RATE:

1. Increase buffer pool
   More memory = more cache space

2. Better data layout
   Store related data together (clustering)

3. Minimize random access
   Use indexes for sequential access

4. Reduce I/O operations
   Use joins wisely

5. Prefetching
   Read ahead of demand

REAL METRICS:

Good database system: 99% hit rate
- 1 miss per 100 queries
```

Poor database system: 40% hit rate

- 60 misses per 100 queries

SLOWDOWN: 100× difference!

This is why:

- Database admins obsess over cache
- Memory is expensive but worth it
- Query optimization = maximize cache hits

Q16: How do you handle disk failures in RAID 5?

Answer:

RAID 5 failure recovery process:

NORMAL OPERATION:

Disk 1	Disk 2	Disk 3	Disk 4
Block1	Block2	Block3	Parity1
Block5	Parity2	Block4	Block6

DISK 2 FAILS (e.g., head crash):

Disk 1	XXXXXX	Disk 3	Disk 4
Block1	?	Block3	Parity1
Block5	?	Block4	Block6

SYSTEM RESPONSE (automatic):

1. Failure Detection (immediate)
 - RAID controller detects disk failure
 - Array switches to DEGRADED mode
 - Alert sent to administrator
2. User Impact (minimal)
 - Requests still work
 - Parity blocks recalculate lost data
 - Performance drops (now 3 disks instead of 4)
 - Some queries may be slower
3. Reconstruction (background, over hours)
 - Spare disk is activated
 - Lost blocks calculated and written:
Block5 = Block1 XOR Block4 XOR Parity2
 - All lost blocks recovered
 - Array back to NORMAL
4. Full Rebuild (after replacement)
 - Failed disk physically replaced
 - Rebuild copies reconstructed data to new disk
 - Takes 24-48 hours

- During rebuild: One more disk failure = TOTAL LOSS
(This is why some use RAID 6)

TIMELINE:

T=0 Disk fails
T=0 Alert sent
T=1min Spare activated
T=6hr Rebuild complete
T=8hr New disk installed
T=32hr Full redundancy restored

USER IMPACT:

- First 1min: Automatic failover
- Next 6 hours: Slightly slower performance
- After 8 hours: Fully redundant again

WORST CASE: Second disk fails during rebuild

- If RAID 5: TOTAL LOSS - all data gone
- If RAID 6: Still recoverable (two parity blocks protect you)

This is why enterprises prefer RAID 6 + hot spares

Q17: What's the difference between hot, warm, and cold data?

Answer:

Business classification for storage tiering:

HOT DATA:

Definition: Frequently accessed, changes often

Storage: SSD or fast disk

Cost per GB: High (\$0.10-0.20/GB)

Access time: <10ms

Retention: Days to weeks

Example: Today's sales, active users, current orders

Real size: 10% of total data

Real importance: 90% of queries

WARM DATA:

Definition: Occasionally accessed, some changes

Storage: Standard HDD or mid-tier SSD

Cost per GB: Medium (\$0.05-0.10/GB)

Access time: 20-100ms

Retention: Weeks to months

Example: Last month's sales, inactive users, past orders

Real size: 30% of total data

Real importance: 9% of queries

COLD DATA:

Definition: Rarely accessed, almost no changes

Storage: Archive storage, magnetic tape

Cost per GB: Very low (\$0.001-0.01/GB)

Access time: Seconds to minutes

Retention: Years (compliance requirement)

Example: Historical data, deleted users, completed projects

Real size: 60% of total data

Real importance: 1% of queries

AUTOMATIC TIERING EXAMPLE:

Day 1: Customer order created

→ Placed on SSD (HOT)

Week 1: Customer viewing order frequently

→ Stays on SSD

Month 1: Customer stopped checking

System auto-moves to HDD (WARM)

Year 1: Old order rarely accessed

System auto-moves to tape (COLD)

Year 8: Legal hold expires

Deleted (after compliance period)

COST COMPARISON:

Without tiering (all SSD):

10TB × \$0.15 = \$1,500

With tiering:

HOT (10% = 1TB): 1 × \$0.15 = \$150

WARM (30% = 3TB): 3 × \$0.07 = \$210

COLD (60% = 6TB): 6 × \$0.005 = \$30

Total: \$390

SAVINGS: 74% cheaper!

PERFORMANCE: Same for active queries (they hit SSD)

Real-world example: Google

- Uses automatic tiering heavily
- Keeps only needed data fast
- Saves millions on storage costs

Q18: How does write-ahead logging protect data?

Answer:

WAL is critical for consistency during crashes:

SCENARIO WITHOUT WAL: Transfer \$200

Execute:

1. Read checking: \$1,000
2. Write checking = \$800 (in memory, not on disk yet)
3. CRASH! Power failure before disk write

Problem: Checking shows \$1,000 again at restart

Savings never got \$200

Money disappeared

SCENARIO WITH WRITE-AHEAD LOGGING:

Execute:

1. Write to log: "Begin transfer"
2. Wait for log disk confirm
3. Read checking: \$1,000
4. Write to log: "Debit checking"
5. Wait for log disk confirm
6. Update checking = \$800 (in memory)
7. Write to log: "Credit savings"
8. Wait for log disk confirm
9. Update savings = \$700 (in memory)
10. Write to log: "Commit transfer"
11. Wait for log disk confirm

CRASH AT STEP 3:

Log shows: "Begin transfer" (incomplete)

Recovery: Ignore, no changes applied

Result: Consistent (original state)

CRASH AT STEP 8:

Log shows: "Begin", "Debit checking", "Credit savings" (incomplete)

Recovery: Redo from log

- Debit \$200 from checking
- Credit \$200 to savings

Result: Consistent (transfer completed)

CRASH AT STEP 11:

Log shows: "Begin", "Debit", "Credit", "Commit"

Recovery: Already done

Result: Consistent (transfer already applied)

KEY INSIGHT: Log written BEFORE changes applied

If crash: Log tells recovery what to do

- Redo: Complete partial transactions
- Undo: Roll back failed transactions

This is why:

- Databases always have transaction logs
- Log disk is often on separate physical disk (RAID 1)

- Log writes happen synchronously (wait for disk)
- Data writes can be asynchronous

Q19: What's the impact of index fragmentation on performance?

Answer:

Over time, indexes become scattered on disk:

IDEAL INDEX (New):

Block 1: [Index nodes 1-10]

Block 2: [Index nodes 11-20]

Block 3: [Index nodes 21-30]

Sequential reads: Block 1 → Block 2 → Block 3

FRAGMENTED INDEX (After updates):

Block 1: [Index nodes 1-5, 25-30]

Block 100: [Index nodes 6-10]

Block 50: [Index nodes 11-15]

Block 2: [Index nodes 16-20]

Block 75: [Index nodes 21-24]

Sequential reads: Block 1 → Block 100 → Block 50 → Block 2 → Block 75

PERFORMANCE IMPACT:

Range query: "Find employees 10,000-20,000"

Unfragmented:

- Block 1 read
- Block 2 read
- Total: 2 disk reads × 5ms = 10ms

Fragmented:

- Block 1 read
- Seek to Block 100
- Block 100 read
- Seek to Block 50
- Block 50 read
- Seek to Block 2
- Block 2 read
- Seek to Block 75
- Block 75 read
- Total: Multiple seeks + 5 reads = 50ms

SLOWDOWN: 5× slower!

CAUSES OF FRAGMENTATION:

1. Insertions in middle of key range

2. Deletions leaving gaps
3. Rebuild of index structure
4. System crashes during writes
5. Multiple simultaneous updates

SOLUTION: Rebuild Index

BEFORE REBUILD:

Fragmented index across many blocks

Range query takes 50ms

REBUILD PROCESS:

1. Read all index entries
2. Sort by key
3. Write sequentially to new blocks

AFTER REBUILD:

Sequential layout

Range query takes 10ms

Typically done:

- Nightly (low-usage time)
- After major data loads
- When performance degrades

Q20: How does SSD wear-out affect long-term database storage?

****Answer:****

SSDs have limited write cycles:

SSD TECHNOLOGY BASICS:

Flash memory = Billions of transistors

Each transistor stores charge (1 bit)

To write: Apply high voltage to set charge

To erase: Apply different voltage to remove charge

To read: Measure if charge present

WEAR: Each erase cycle damages transistor slightly

After ~3,000-5,000 erase cycles: Transistor fails

COMPARISON TO HDD:

HDD: Mechanical component

Can operate 100+ years theoretically

Wear: Physical degradation of head/platter

Failure: Sudden (head crash)

Warning: Some advance warning (clicking sounds)

SSD: Electronic component

Rated 5-7 years (typical)

Wear: Voltage stress on transistors

Failure: Gradual (bit errors increase)

Warning: Can predict failure (error rates)

WRITE AMPLIFICATION:

Database writes: 1 page = 4KB to SSD

SSD internally:

1. Read entire block (e.g., 256KB)
2. Update relevant 4KB page in block
3. Erase entire block
4. Write entire block back

ACTUAL I/O: 256KB written for 4KB change!

Write amplification factor: 64×

High-write workloads kill SSDs quickly:

LIFESPAN CALCULATION:

SSD spec: 100,000 endurance (total writes before failure)

Example: 1TB SSD = 100,000 DWPDP (Drive Writes Per Day)

DWPDP means: Can write entire drive 100,000 times before failure

Usage scenario 1 (Light):

- 100GB written per day
- $100\text{GB} / 1\text{TB} = 0.1$ DWPDP
- Lifespan: $100,000 / 0.1 = 1,000,000$ days $\approx 2,700$ years
- Practical: 10-15 years (other factors degrade it)

Usage scenario 2 (Heavy database):

- 1TB written per day
- $1\text{TB} / 1\text{TB} = 1$ DWPDP
- Lifespan: $100,000 / 1 = 100,000$ days ≈ 274 years
- Sounds great but...

Usage scenario 3 (Very heavy with write amplification):

- 1TB written per day (user data)
- 64TB actual flash writes (write amplification)

- 64TB / 1TB = 64 DWPD
- Lifespan: 100,000 / 64 = 1,562 days $\approx$ 4.3 years

MITIGATION:

1. Use enterprise SSDs
 - Better wear leveling
 - Over-provisioning
 - More endurance
2. Separate write-heavy and read-only data
 - Logs: HDD (lots of writes)
 - Data: SSD (mostly reads)
3. Implement write coalescing
 - Batch writes
 - Reduce number of erase cycles
4. Monitor SSD health
 - SMART errors
 - Bit error rates
 - Proactive replacement

REAL EXAMPLE:

Database with 500GB/day writes:

Traditional SSD: 4 year lifespan

Cost per year: \$250/year = \$1,000 total

Enterprise SSD with better spec: 10 year lifespan

Cost per year: \$350/year = \$3,500 total

But: Enterprise SSD gets 2.5 $\times$ more years

Effective cost: \$350/year still better long-term

Plus: Enterprise SSD survives unexpectedly high write loads

PRESENTATION DELIVERY TIPS

Structure Your Presentation (45 minutes)

Opening (5 min)

- Start with impact: "Every database access to disk is 1,000,000 $\times$ slower than CPU"
- Show real example (e.g., Google search involving disk)
- Outline topics

****Part 1: Storage Fundamentals (10 min)****

- Memory hierarchy
- Why secondary storage matters
- HDD vs SSD comparison
- Show diagrams of disk structure

****Part 2: File Organization (12 min)****

- Live demo: Show how records fit in blocks
- Explain heap vs sorted vs hash
- Use comparison table
- Give pros/cons for each

****Part 3: Advanced Topics (13 min)****

- RAID levels with visual examples
- Modern architectures brief overview
- Real-world case studies

****Closing (5 min)****

- Key takeaways
- Connection to exam topics
- Q&A preparation

Engage Your Audience****Use Concrete Examples:****

- "Netflix video streaming = stored on RAID 0 for speed"
- "Bank database = RAID 1 for zero data loss"
- Real numbers ("1 disk read = 115 days of CPU work")

****Interactive Elements:****

- "Who can guess how many employee records fit in 4KB block?"
- "What happens if disk fails during RAID rebuild?"
- Show live calculations of access times

****Visuals:****

- Disk diagram with rotating platter
- Tree structure of B-tree
- RAID configurations side-by-side
- Timeline of tiering data

Common Student Questions to Prepare For

1. "Why do we care about old technology if we use the cloud?"
 - *Answer: Cloud storage still uses same principles*
2. "Isn't SSD always better than HDD?"
 - *Answer: Cost, capacity, write limitations matter*
3. "How does the OS handle all this?"
 - *Answer: Transparent buffering, OS chooses strategies*
4. "Do I need to know exact seek times for exam?"
 - *Answer: Understand concepts, not memorize numbers*

KEY FORMULAS FOR QUICK REFERENCE

Blocking Factor = Block Size / Record Size

Access Time = Seek Time + Rotational Delay + Transfer Time

Number of Blocks = Total Records / Blocking Factor

Transfer Rate = Block Size / Transfer Time

Cache Hit Ratio = Cache Hits / Total Requests

Disk Utilization = Total Disk Operations / Max Possible Operations

RAID Capacity = $(n-p) \times \text{Disk Size}$

where n = number of disks, p = parity disks

SUMMARY

Your lecture should emphasize:

1. **Storage is the bottleneck** - CPU speed far outpaces disk speed
2. **File organization matters** - Choice affects query performance by orders of magnitude
3. **RAID is essential** - Reliability and performance both important
4. **Trade-offs everywhere** - Speed vs cost, capacity vs access time
5. **Modern systems are sophisticated** - Automatic tiering, caching, smart scheduling
6. **Understanding principles helps** - Whether using traditional databases or cloud storage

GOOD LUCK WITH YOUR PRESENTATION! 🎓

You now have deep understanding of every concept in Chapter 16. Your classmates will appreciate the real-world examples and connections to their experience.